Assignment 2

SCE-128 Technical Report

Design of a Vector Embedding for Capability Composition

Technical Report and Experimental Evaluation of SCE-128 v2.1

Course / Assignment:	PCCST503 - Assignment 2
Submitted by:	Madhav Krishna
University Reg. No:	TCR24CS043
Report date:	02 October 2026
Final model:	SCE-128 v2.1
Vector dimension:	128
Capability records:	18

Abstract

This report presents SCE-128, a problem-specific vector representation for formally specified application states, goals, and executable capabilities. The design is motivated by the Assignment 2 requirement to preserve functional identity, state awareness, precondition-effect compatibility, input-output compatibility, composability, goal relevance, and operational attributes. Instead of treating a capability as undifferentiated text, SCE-128 encodes typed structural blocks for function, type, inputs, outputs, preconditions, effects, constraints, resources, execution mechanism, and operational properties. A separate symbolic relation layer evaluates directional compatibility, while a hybrid goal-relevance layer combines embedding evidence with backward symbolic reachability. The final composition component is a leakage-free learned composer that receives atomic capability vectors and vector-derived relation features, while the independently encoded formal composite is used only as an evaluation target. On the constructed e-commerce capability dataset, the final model achieves a compatibility-proxy AUC of 0.9000, a held-out direct composition mean of 0.8546, a recursive-chain consistency mean of 0.8316, and a goal-relevance AUC of 0.9722. Controlled operational perturbations produce a mean operational-block cosine of 0.7714 while the non-operational representation remains at 1.0000 cosine. The main limitation is scope: the dataset is small and domain-specific, so the results establish feasibility and internal consistency rather than general-purpose embedding performance.

Keywords: capability embedding, structured vector representation, capability composition, compatibility, preconditions, effects, goal relevance, operational attributes

Contents

Abstract **2**

1 Problem Definition **5**

1.1 Application domain . . . 5
1.2 Research question . . . 5
1.3 Scope boundary . . . 5

2 Design Requirements **5**

3 Related Embedding Approaches **6**

4 Proposed Representation **7**

4.1 Final architecture . . . 7
4.2 Formal capability representation . . . 7
4.3 128-dimensional capability vector . . . 7
4.4 State and goal representations . . . 7

5 Mathematical Formulation **8**

5.1 State and goal . . . 8
5.2 Predicate encoding . . . 8
5.3 Capability embedding . . . 8
5.4 Similarity . . . 9
5.5 Directional compatibility . . . 9
5.6 State applicability and goal relevance . . . 9
5.7 Operational attributes . . . 9

6 Capability Composition Model **9**

6.1 Formal composition . . . 9
6.2 Baseline composition operator . . . 9
6.3 Leakage-free learned composer . . . 10

7 Implementation **10**

7.1 Software organization . . . 10
7.2 Core interfaces . . . 10
7.3 Dependencies and reproducibility . . . 10
7.4 Execution pipeline . . . 11

8 Experimental Methodology **11**

8.1 Composition split . . . 12
8.2 Evaluation principle . . . 12

9 Results **12**
9.1 Final result summary . . . 12
9.2 Composition results . . . 13
9.3 Goal relevance . . . 13
9.4 Operational attributes . . . 14
9.5 Alternative implementations . . . 14

10 Analysis **14**
10.1 Similarity is not composability . . . 14
10.2 Composition quality improves when composition is learned independently . . . 14
10.3 Alternative implementations reveal a trade-off . . . 15
10.4 Goal relevance benefits from symbolic structure . . . 15
10.5 Operational attributes are isolated . . . 15
10.6 Efficiency . . . 15

11 Limitations **15**

12 Conclusion **16**

A Appendix A - Dataset and Capability Inventory **17**

B Appendix B - Execution and Reproduction Guide **17**
4.1 Clone the repository . . . 17
4.2 Create and activate a virtual environment . . . 17
4.3 Install dependencies . . . 18
4.4 Run the complete evaluation pipeline . . . 18
4.5 Run the regression tests only . . . 18
4.6 Run the final integrated evaluation only . . . 18
4.7 Run individual phases . . . 18
4.8 Run the project smoke test . . . 18

C Appendix C - Traceability Matrix **19**

References **20**

1. Problem Definition

The assignment changes the emphasis from planning a sequence of transitions to representing the operations that make up that sequence. The formal application model is defined as $A = (S, C, S_I, G, R, K)$, where S is the state space, C is the set of capabilities, S_I is the initial state, G is the goal specification, R is the available resource set, and K contains global constraints and policies. The assignment explicitly excludes redesign of the BFS, DFS, A*, D* Lite, or LPA* planning algorithms; those algorithms are background context only. [A1]

The central problem is therefore a representation problem: construct a numerical space in which formally specified capabilities can be compared, related, composed, and evaluated against states and goals without reducing capability composition to ordinary semantic similarity. The specification explicitly distinguishes functional similarity from composability, requiring directional relationships such as effects satisfying downstream preconditions and outputs satisfying downstream inputs. [A1]

1.1 Application domain

The implementation uses an e-commerce purchase workflow as the controlled application domain. The main execution chain is deliberately formal and state-based: `AddToCart -> CheckInventory -> ReserveInventory -> CreateOrder -> MakePayment -> VerifyPayment -> SendNotification`. The domain is compact enough for controlled experiments but rich enough to contain positive compositions, incompatible transitions, alternative implementations, irrelevant operations, and operational variations.

1.2 Research question

> *How can formally specified states, goals, and executable capabilities be represented in a vector space such that the representation preserves the relationships required for compatibility, composition, and construction of complex application functionality?*

This wording follows the primary research question given in the assignment specification. [A1]

1.3 Scope boundary

- The project represents formal states, goals, and executable capabilities.
- The project evaluates similarity, directional compatibility, goal relevance, operational attributes, and composition.
- The project does not redesign the Assignment 1 planner or replanning algorithms.
- The final model is evaluated on a small, formally constructed e-commerce dataset rather than a broad natural-language corpus.

2. Design Requirements

The implementation was designed directly against the requirements in the assignment specification. The capability tuple is required to account for capability type, inputs, outputs, preconditions, effects, capability-specific constraints, resources, quality attributes, reliability, availability, and execution mechanism. [A1]

Requirement	Interpretation	Implementation response
Capability identity	Different functions must be distinguishable.	Function/type/input/output/effect blocks; capability ID excluded from vector.

Requirement	Interpretation	Implementation response
State awareness	Relate capabilities to the states on which they operate.	Shared predicate representation plus state-applicability score.
Precondition-effect compatibility	Capture $E_i \Rightarrow P_j$ relationships.	Directional effect-to-precondition matching and symbolic checks.
Input-output compatibility	Represent data dependencies across capabilities.	Typed output-to-input matching.
Similarity vs composability	Similar capabilities need not compose.	Separate cosine/block similarity from directional compatibility.
Composition	Represent $C_i \circ C_j$ and longer chains.	Formal composition plus leakage-free learned vector composer.
Goal relevance	Relate capabilities to desired goal states.	Effect-goal alignment plus symbolic backward reachability.
Operational properties	Represent cost, reliability, availability, risk, resources.	Dedicated operational 20-D block and controlled perturbation experiments.
Consistency	Behave consistently across problems.	Formal dataset validation and regression tests.
Efficiency	Keep computational/storage requirements reasonable.	Small structured encoder; no external pretrained embedding service.

A key design constraint is that experimental labels such as compatible, irrelevant, or alternative implementation are used as ground truth for evaluation and are not directly inserted into the capability vector. This prevents direct label leakage into the representation.

3. Related Embedding Approaches

The assignment motivates its problem from the success of distributed word representations such as Word2Vec. The project therefore reviews representative embedding families and identifies the gap that motivates SCE-128.

Approach	Reference	Core idea	Gap / relevance to this assignment
Word2Vec / Skip-gram	Mikolov et al.	Learns distributed word/phrase representations and semantic/syntactic relationships.	Strong semantic geometry; not designed to encode typed capability contracts or directional state transitions.
GloVe	Pennington et al.	Learns word vectors from global corpus co-occurrence statistics.	Useful semantic representation but still primarily distributional; precondition/effect direction is not explicit.
Sentence-BERT	Reimers & Gurevych	Produces sentence-level embeddings intended for semantic similarity and retrieval.	More expressive semantic similarity, but formal capability fields and typed composition relations remain implicit.

Approach	Reference	Core idea	Gap / relevance to this assignment
node2vec	Grover & Leskovec	Learns node representations from graph neighborhoods.	Useful for dependency graphs; standard node embeddings do not by themselves encode typed inputs, outputs, preconditions, effects, or operational attributes.
SCE-128	This work	Structured field-wise vectors + symbolic relations + learned composition.	Explicitly separates functional attributes, directional compatibility, goal relevance, and operational properties.

The design decision is therefore not to reject general-purpose embeddings, but to avoid using one as the complete solution. The assignment itself states that merely applying an existing embedding model is insufficient. [A1] SCE-128 instead adapts the vector-space idea to the formal structure of executable capabilities.

4. Proposed Representation

4.1 Final architecture

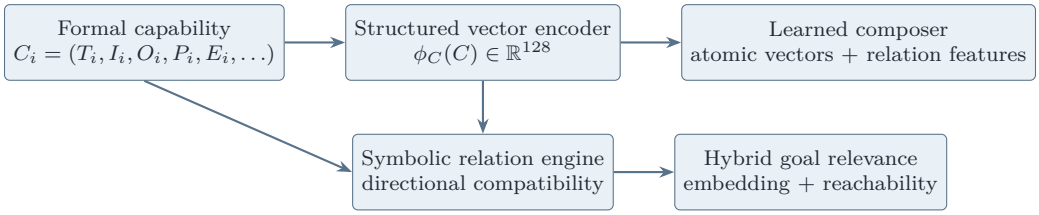


Figure 1: SCE-128 v2.1 architecture: structured vector representation, symbolic compatibility, hybrid goal relevance, and learned composition.

The final architecture is intentionally hybrid. The vector encoder provides a compact, numerical representation; the symbolic relation engine retains exact directional logic; and the learned composer maps atomic vector pairs and vector-derived relation features into predicted composite vectors. This avoids treating cosine similarity as a substitute for logical compatibility.

4.2 Formal capability representation

$$C_i = (T_i, I_i, O_i, P_i, E_i, K_i, R_i, Q_i, Rel_i, A_i, M_i)$$

Here T is capability type, I and O are inputs and outputs, P and E are preconditions and effects, K contains constraints, R contains resources, Q contains cost/quality attributes, Rel is reliability, A is availability, and M is the execution mechanism. This is the core tuple specified by the assignment. [A1]

4.3 128-dimensional capability vector

The final vector is the concatenation of structured sub-representations followed by L2 normalization. The capability identifier itself is metadata only and is deliberately excluded from the vector.

4.4 State and goal representations

A state is represented as a set of attribute-value predicates. A goal is represented as a set of desired predicates. Both use the same underlying predicate representation so that a state fact such as `Payment.status =`

Vector block	Dimensions
Function	4
Type	8
Input	16
Output	16
Precondition	16
Effect	16
Constraint	12
Resource	12
Mechanism	8
Operational	20

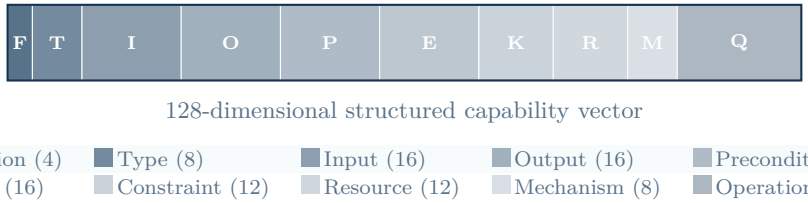


Figure 2: SCE-128 block allocation across the formal capability tuple.

`SUCCESS` and a goal predicate with the same formal condition occupy a common comparison space. This satisfies the assignment requirement for state awareness and goal relevance without converting the formal model into free-form natural language. [A1]

5. Mathematical Formulation

5.1 State and goal

$$S = \{(x_1, v_1), (x_2, v_2), \ldots, (x_n, v_n)\}$$
$$G = \{g_1, g_2, \ldots, g_m\}, \quad S \models G$$

5.2 Predicate encoding

Each formal condition is canonicalized as $p = (a, o, v)$, where a is the attribute, o is the operator, and v is the value. The value representation distinguishes categorical, Boolean, and numeric values while preserving the attribute identity and operator. Set-valued fields are pooled with a permutation-invariant mean followed by a learned projection so that ordering of conditions within a set does not affect the encoding.

$$e_p = \mathrm{MLP}_p[e_a \,||\, e_o \,||\, e_v]$$
$$\mathrm{SetEnc}(X) = \rho\left(\frac{1}{|X|}\sum_{x \in X} \psi(x)\right)$$

5.3 Capability embedding

$$\phi_C(C) = \mathrm{Norm}(h_T \,||\, h_I \,||\, h_O \,||\, h_P \,||\, h_E \,||\, h_K \,||\, h_R \,||\, h_M \,||\, h_Q) \in \mathbb{R}^{128}$$
$$\mathrm{Norm}(x) = x/\|x\|_2$$

5.4 Similarity

$$\mathrm{Sim}(C_i, C_j) = \cos(\phi_C(C_i), \phi_C(C_j))$$

Whole-vector cosine is treated as a descriptive similarity measure. It is not the authoritative composition criterion.

5.5 Directional compatibility

$$M_{PE}(C_i, C_j) = \frac{1}{|P_j|} \sum_{p \in P_j} \max_{e \in E_i} \cos(U_E e, U_P p)$$

$$M_{IO}(C_i, C_j) = \frac{1}{|I_j|} \sum_{i \in I_j} \max_{o \in O_i} \cos(U_O o, U_I i)$$

$$\mathrm{Compat}(C_i, C_j) = \mathrm{sigmoid}(b + w_1 M_{PE} + w_2 M_{IO} - w_3 \mathrm{Conflict})$$

The symbolic engine additionally checks exact formal compatibility. This dual approach is necessary because two capabilities can be semantically similar while being unable to execute sequentially.

5.6 State applicability and goal relevance

$$\mathrm{Applicable}(C, S) = \frac{1}{|P_C|} \sum_{p \in P_C} \max_{s \in S} \cos(U_P p, U_S s)$$

$$\mathrm{GoalRel}_{sem}(C, G) = \frac{1}{|G|} \sum_{g \in G} \max_{e \in E_C} \cos(U_E e, U_G g)$$

The final goal-relevance score adds exact goal-support and backward symbolic reachability to the semantic effect-goal signal. Therefore the final metric is a hybrid score, as reported in the integrated evaluation.

5.7 Operational attributes

$$q = [\log(1+t),\ \text{resource},\ \log(1+\text{money}),\ \text{risk},\ \text{energy},\ \text{reliability},\ \text{availability}]$$

$$h_Q = \mathrm{MLP}_Q(\mathrm{Norm}(q)) \in \mathbb{R}^{20}$$

Controlled experiments change one operational property while holding all non-operational fields constant. This provides a direct isolation test for the operational block.

6. Capability Composition Model

6.1 Formal composition

$$C_{ij} = C_j \circ C_i$$

For sequential capabilities, the composite inputs contain externally required inputs after removing inputs provided internally by the first capability. Composite preconditions remove conditions satisfied by earlier effects. Composite effects are the net state changes under sequential execution. Operational quantities such as execution time and monetary cost are aggregated under the stated sequential assumptions; reliability is multiplied under the simplifying independence assumption used by the implementation.

6.2 Baseline composition operator

$$v_{ij}(\text{base}) = 0.45 v_i + 0.45 v_j + 0.10 r_{ij}$$

This fixed operator is retained as a baseline rather than presented as the final solution. Its main weakness is composition drift over longer chains.

6.3 Leakage-free learned composer

$$
\begin{aligned}
z_{ij} &= [v_i \,||\, v_j \,||\, r_{IO} \,||\, r_{PE}] \\
\hat{v}_{ij} &= \text{Normalize}(\text{MLP}_{comp}(z_{ij})) \\
L_{comp} &= 1 - \cos(\hat{v}_{ij}, v^*_{ij})
\end{aligned}
$$

The predicted composite vector $\hat{v}_{ij}$ is generated from atomic vectors and vector-derived relation features only. The target v^*_{ij} is obtained by independently encoding the formal composite. The final composition dataset uses 8 training pairs, 4 held-out direct pairs, and 9 evaluation chains. This construction prevents the model from receiving the formal composite as an input during prediction.

7. Implementation

7.1 Software organization

```
SCE128_Capability_Embedding_Final/
  data/
    core_application.json
    expanded_application.json
    composition_split.json
  src/
    schema.py
    data_models.py
    dataset.py
    encoder.py
    compatibility.py
    formal_composition.py
    final_model.py
    learned_composer.py
  experiments/
    phase01_dataset_validation.py
    phase02_capability_compatibility.py
    phase03_baseline_composition.py
    phase04_alternative_implementations.py
    phase05_goal_relevance.py
    phase06_operational_attributes.py
    phase07_leakage_free_composition.py
    phase08_final_evaluation.py
  tests/
  run_all.py
  requirements.txt
```

7.2 Core interfaces

```
encode(state)
encode(goal)
encode(capability)
compose(capabilities)
similarity(x, y)
```

These interfaces implement the functional API requested by the assignment. The final project also separates symbolic composition from vector composition so that the ground-truth formal model remains inspectable.

7.3 Dependencies and reproducibility

- Python 3.x; the captured run used Python 3.14.3 on Windows 10/11 PowerShell.

- NumPy for numerical representation and vector operations.
- pytest for automated regression tests.
- No external pretrained embedding service is required.
- The cleaned project package is approximately 416 KB and contains the complete source, data, tests, and experiment drivers.

7.4 Execution pipeline

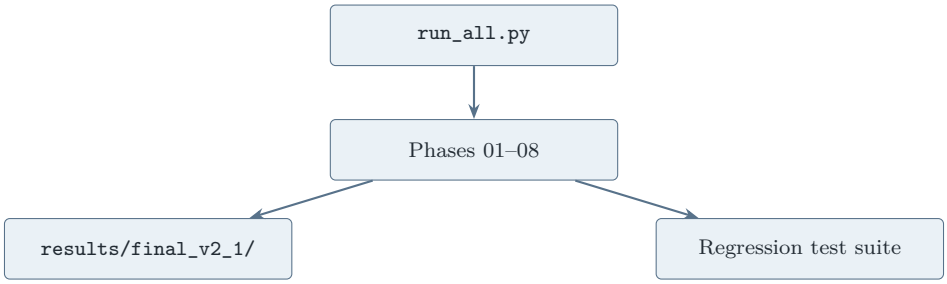


Figure 3: Execution architecture used by the final pipeline.

8. Experimental Methodology

The experiments use formally specified application data rather than natural-language descriptions. Each experiment has an explicit ground-truth interpretation derived from the formal state, goal, capability, or composition specification. Evaluation labels are withheld from the vector encoder and are used only for measurement.

Phase	Experiment	Dataset	Metrics
Phase 01	Formal dataset validation	18 capability records; 15 logical + 3 alternative implementations.	ID uniqueness, required relation accuracy, chain validity, implementation contract checks.
Phase 02	Capability compatibility	13 directed labeled pairs: 8 positive and 5 negative.	Accuracy, precision, recall, F1, raw cosine AUC, compatibility-proxy AUC.
Phase 03	Baseline composition	2 reference chains.	Prefix and final vector consistency; goal alignment; operational totals.
Phase 04	Alternative implementations	API, DB, GUI variants of CreateOrder plus negative controls.	Functional similarity, implementation-block similarity, overall similarity gap.
Phase 05	Goal relevance	12 relevant and 6 irrelevant capabilities.	Mean score gap, AUC, Precision@5, Precision@10, ranking inspection.
Phase 06	Operational attributes	4 controlled perturbations.	Whole-vector, operational-block, and non-operational-block cosine.
Phase 07	Leakage-free composition	8 training pairs, 4 held-out direct pairs, 9 recursive chains.	Held-out direct mean and recursive chain consistency.

Phase	Experiment	Dataset	Metrics
Phase 08	Final integrated evaluation	All final metrics.	Consolidated model evaluation and regression checks.

8.1 Composition split

- Training pairs: 8.
- Held-out direct pairs: 4.
- Recursive evaluation chains: 9.
- The formal composite is excluded from the prediction input and appears only as the evaluation target.

8.2 Evaluation principle

The study intentionally reports both successful and weak cases. A result is not treated as a success merely because a single metric is high; the analysis also checks whether the metric measures the intended phenomenon and whether any part of the architecture is using leaked ground-truth information.

9. Results

9.1 Final result summary

Area	Metric	Final result
Compatibility	Formal accuracy	1.0000
Compatibility	Raw cosine AUC	0.6250
Compatibility	Compatibility-proxy AUC	0.9000
Composition	Held-out direct mean	0.8546
Composition	Recursive-chain mean	0.8316
Composition	Minimum recursive consistency	0.7929
Alternative implementations	Functional similarity gap	0.0932
Alternative implementations	Overall similarity gap	0.0569
Alternative implementations	Mean implementation-block cosine	0.0265
Goal relevance	Relevant mean score	0.7552
Goal relevance	Irrelevant mean score	0.4404
Goal relevance	AUC	0.9722
Goal relevance	Precision@5	1.0000
Goal relevance	Precision@10	1.0000
Operational attributes	Mean whole-vector cosine	0.9771
Operational attributes	Mean operational-block cosine	0.7714
Operational attributes	Mean non-operational cosine	1.0000
Software quality	Regression tests	21/21 passed

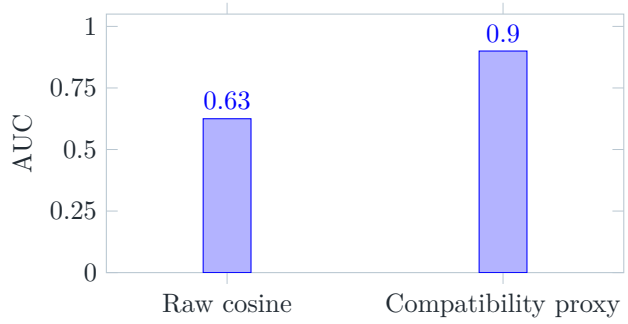


Figure 4: Experiment 1: raw cosine similarity is weaker than the explicit compatibility proxy.

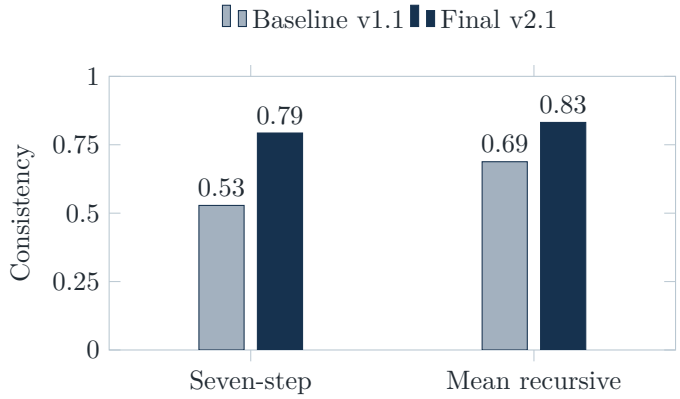


Figure 5: Seven-step and mean recursive composition consistency: baseline v1.1 versus final v2.1.

9.2 Composition results

The baseline seven-step purchase chain has final consistency 0.5283, while the leakage-free learned composer reaches 0.7929 on the same chain. Across nine recursive evaluation chains, the final mean consistency is 0.8316 and the minimum is 0.7929. Held-out direct-pair consistency averages 0.8546. The training mean of 0.8531 is close to the held-out mean, which does not prove generalization but provides no obvious training-to-test collapse.

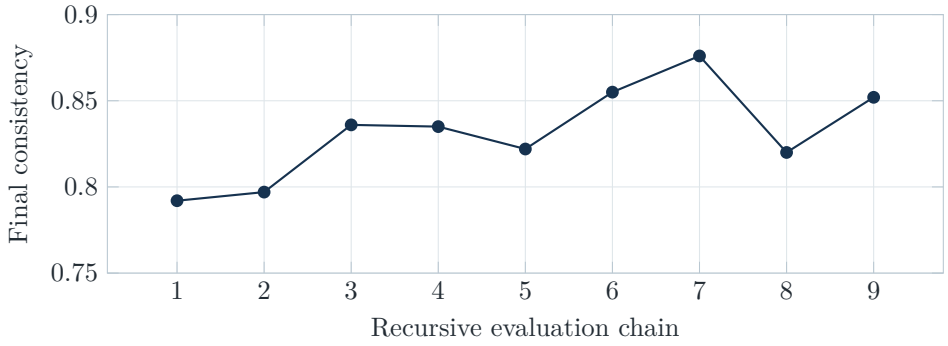


Figure 6: Final consistency across nine recursive evaluation chains.

9.3 Goal relevance

The final hybrid goal-relevance score separates the labeled relevant and irrelevant groups by 0.3148 mean score and reaches AUC 0.9722. Precision@5 and Precision@10 are both 1.0000 on the constructed ranking set.

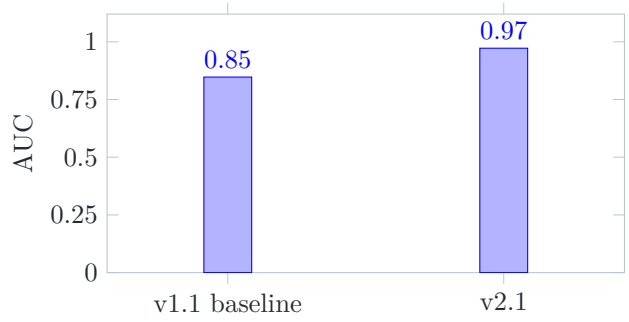


Figure 7: Goal-relevance AUC: v2.1 versus the v1.1 baseline.

9.4 Operational attributes

Controlled perturbations show that operational changes primarily affect the operational block. The unavailable variant has the strongest change, with operational-block cosine 0.3554, while the mean non-operational cosine across all four variants remains 1.0000.

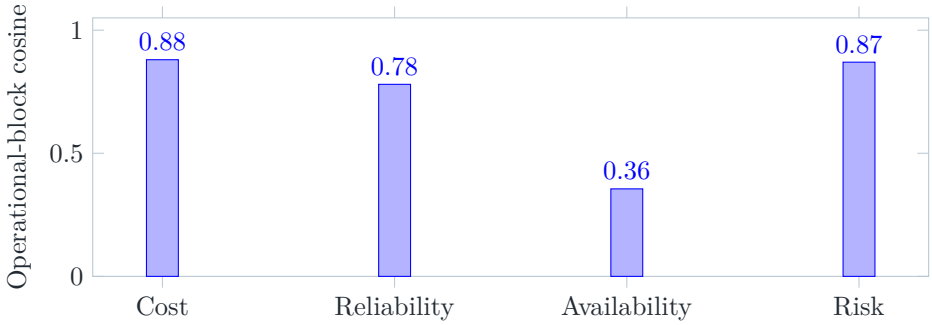


Figure 8: Operational-block response to controlled changes in cost, reliability, availability, and risk.

9.5 Alternative implementations

Pair	Functional	Implementation	Operational	Overall v2.1
C4_API ↔ C4_DB	0.8150	-0.0627	0.9990	0.7365
C4_API ↔ C4_GUI	0.7264	0.0373	0.9919	0.6707
C4_DB ↔ C4_GUI	0.8585	0.1049	0.9909	0.7898

10. Analysis

10.1 Similarity is not composability

Experiment 1 provides direct evidence for the assignment's distinction between similarity and composability. Raw whole-vector cosine yields AUC 0.6250, while the explicit compatibility proxy reaches AUC 0.9000. The formal compatibility engine itself reproduces all 13 labeled relations exactly. Therefore, vector similarity can be useful descriptive evidence, but the directional structure of effects, preconditions, outputs, and inputs carries most of the composability signal.

10.2 Composition quality improves when composition is learned independently

The first deterministic composition baseline drifts substantially on longer chains. The leakage-free v2.1 composer improves the seven-step chain from 0.5283 to 0.7929 and reaches a recursive-chain mean of 0.8316. The improvement is especially meaningful because the predicted vector is produced without supplying the

formal composite. The result should still be treated as domain-specific evidence because the training set contains only eight direct pairs and the evaluation dataset is small.

10.3 Alternative implementations reveal a trade-off

API, database, and GUI implementations preserve the same functional contract, but their final overall vectors are only moderately separated from negative controls. The final functional similarity gap is 0.0932 and the overall gap is 0.0569. Compared with the v1.1 baseline, the final model sacrifices some functional-cluster tightness: the baseline variants had functional mean 1.0000, whereas the final mean is approximately 0.8000. This is not a failure of the experiment; it reveals a genuine trade-off between preserving implementation distinction and forcing exact functional identity in a single overall geometry.

10.4 Goal relevance benefits from symbolic structure

The final goal-relevance score reaches AUC 0.9722, substantially above the v1.1 baseline AUC of 0.8472. The improvement comes from the hybrid use of backward symbolic reachability in addition to effect-goal similarity. This design reduces false positives such as a cancellation operation that is semantically related to an order but does not lie on a path to the selected success goal. The result demonstrates the value of combining vector representations with the application structure instead of expecting a single cosine score to encode all planning semantics.

10.5 Operational attributes are isolated

The operational experiment is one of the cleanest because it uses controlled perturbations. All non-operational fields remain unchanged and their cosine remains 1.0000. The operational block, however, responds differently to each change, with the unavailable condition producing the largest shift. This supports the design choice of keeping operational attributes in a dedicated block rather than distributing them indistinguishably across the entire representation.

10.6 Efficiency

The captured Windows run completed the eight-phase evaluation pipeline in approximately 4.3 seconds, plus about 0.63 seconds for the regression suite, for a total of roughly 4.9 seconds in the recorded environment. These values are environment-specific, but they demonstrate that the final implementation is computationally lightweight for the current dataset. The cleaned project is approximately 416 KB and depends only on NumPy and pytest.

11. Limitations

1. **Small dataset.** The final evaluation uses 18 capability records in one e-commerce domain. This is sufficient for controlled experiments but not for establishing broad generalization.
2. **Domain dependence.** Attribute names, state variables, resources, and workflow relationships are application-specific. A second domain would be needed to test transfer.
3. **Hybrid reasoning.** Goal relevance uses symbolic backward reachability in addition to vector evidence, so the AUC should not be interpreted as pure embedding-only performance.
4. **Limited composition training data.** The learned composer uses 8 training pairs. Larger and more diverse composition datasets are required to test more complex generalization.
5. **Implementation trade-off.** The final model separates alternative mechanisms better overall but is less tightly clustered in the functional subspace than the v1.1 baseline.
6. **Operational assumptions.** Composite reliability uses a simplifying independence assumption, and operational attributes are controlled synthetic values rather than measurements from a production system.
7. **No large external language corpus.** The design intentionally avoids a pretrained language model because the assignment requires a problem-specific representation; this also means the model does not

inherit broad world knowledge.

12. Conclusion

This project developed and evaluated SCE-128 v2.1, a structured vector representation for formally specified application states, goals, and executable capabilities. The representation preserves explicit blocks for capability function, type, inputs, outputs, preconditions, effects, constraints, resources, execution mechanism, and operational attributes. A separate symbolic compatibility layer provides directional reasoning that raw vector similarity alone cannot supply, while a hybrid goal-relevance layer combines vector evidence with formal reachability.

The final evaluation demonstrates that the design can distinguish composability from similarity, encode operational attributes without disturbing non-operational structure, preserve relationships across alternative implementations, rank goal-relevant capabilities, and construct composite vector representations with a leakage-free learned operator. The most important composition result is a held-out direct mean of 0.8546 and a recursive-chain mean of 0.8316, with the seven-step purchase chain reaching 0.7929. The main conclusion is therefore not that SCE-128 is a universal embedding solution, but that a structured and hybrid representation can preserve the functional relationships required by capability composition on a formally specified application domain.

The work provides a foundation for a larger system in which formal application specifications are embedded, capabilities are composed, and planning systems can reason over the resulting representation. Future work should test multiple application domains, expand the composition dataset, and investigate stronger ways to preserve functional identity and implementation distinction simultaneously.

A. Appendix A - Dataset and Capability Inventory

The expanded dataset contains 18 capability records. The primary purchase goal contains 3 formal target predicates. The composition split contains 8 training pairs, 4 held-out direct pairs, and 9 recursive evaluation chains.

ID	Capability	Type	Functional group / Role	Goal relevant?
C1	AddToCart	FUNCTION	- / Logical capability	Yes
C2	CheckInventory	SERVICE	- / Logical capability	Yes
C3	ReserveInventory	SERVICE	- / Logical capability	Yes
C4	CreateOrder	API	CreateOrder / Canonical	Yes
C4_API	CreateOrder_API	API	CreateOrder / Alternative implementation	Yes
C4_DB	CreateOrder_DATABASE	DATABASE	CreateOrder / Alternative implementation	Yes
C4_GUI	CreateOrder_GUI	GUI	CreateOrder / Alternative implementation	Yes
C5	MakePayment	API	- / Logical capability	Yes
C6	VerifyPayment	API	- / Logical capability	Yes
C7	SendNotification	EVENT	- / Logical capability	Yes
C8	RefundPayment	API	- / Logical capability	No
C9	CancelOrder	API	- / Logical capability	No
C10	ReleaseInventory	SERVICE	- / Logical capability	No
C11	RemoveFromCart	API	- / Logical capability	No
C12	UpdateOrder	API	- / Logical capability	Yes
C13	GetOrder	API	- / Logical capability	Yes
C14	UpdateProfile	API	- / Logical capability	No
C15	GenerateSalesReport	COMPUTATION	- / Logical capability	No

B. Appendix B - Execution and Reproduction Guide

4.1 Clone the repository

```
git clone <REPOSITORY_URL>
cd SCE128_Capability_Embedding_Final
```

Replace `<REPOSITORY_URL>` with the URL of your Git repository.

4.2 Create and activate a virtual environment

Using a virtual environment keeps project dependencies isolated from the system Python installation.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

If PowerShell blocks script execution for the current session, use:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

4.3 Install dependencies

```
python -m pip install -upgrade pip
python -m pip install -r requirements.txt
```

4.4 Run the complete evaluation pipeline

This is the recommended command for reproducing the final results. It runs all numbered phases in order, executes the final integrated evaluation, and runs the regression test suite.

```
python run_all.py
```

Results are written to the `results/` directory. The final v2.1 artifacts are stored under:

```
results/final_v2_1/
```

4.5 Run the regression tests only

```
python -m pytest
```

4.6 Run the final integrated evaluation only

Use this when the full dataset and model are already prepared and only the final metrics are required.

```
python -m experiments.phase08_final_evaluation
```

4.7 Run individual phases

Each experiment is available as a Python module and can be executed independently:

```
python -m experiments.phase01_dataset_validation
python -m experiments.phase02_capability_compatibility
python -m experiments.phase03_baseline_composition
python -m experiments.phase04_alternative_implementations
python -m experiments.phase05_goal_relevance
python -m experiments.phase06_operational_attributes
python -m experiments.phase07_leakage_free_composition
python -m experiments.phase08_final_evaluation
```

4.8 Run the project smoke test

```
python -m src
```

The smoke test prints the capability/vector configuration, selected compatibility relationships, goal-relevance examples, learned-composition configuration, and the 128-dimensional block layout.

C. Appendix C - Traceability Matrix

Assignment requirement	Formal element	Report section	Implementation
Formal application model	$A = (S, C, S_I, G, R, K)$	Section 1, 4	`data/core_application.json`; `data/expanded_application.json`
Capability model	$C = (T, I, O, P, E, K, R, Q, R$	Section 4, 5	`src/schema.py`; `src/data_models.py`
Embedding function	ϕ_C in $\mathbb{R}^{128}$	Section 4, 5	`src/encoder.py`; `src/final_model.py`
Similarity / compatibility	cosine + directional relation score	Section 5, Phase 02	`src/compatibility.py`
Composition	formal composition + learned composer	Section 6, Phase 03/07	`src/formal_composition.py`; `src/learned_composer.py`
Implementation API	encode / compose / similarity	Section 7	`src/*`
Compatibility experiment	Phase 02	Section 8/9	`experiments/phase02_capability_compatibility.py`
Composition experiment	Phase 03 and 07	Section 8/9	`experiments/phase03_baseline_composition.py`; `phase07_leakage_free_composition.py`
Alternative implementations	Phase 04	Section 8/9	`experiments/phase04_alternative_implementations.py`
Irrelevant capabilities	Phase 05	Section 8/9	`experiments/phase05_goal_relevance.py`
Operational attributes	Phase 06	Section 8/9	`experiments/phase06_operational_attributes.py`
Final evaluation	Phase 08	Section 9	`experiments/phase08_final_evaluation.py`

References

[A1] PCCST503, Assignment 2. *Design of a Vector Embedding for Capability Composition.* Assignment specification, 13 pages. Sections 1-10.

[1] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems.*

[2] Pennington, J., Socher, R., and Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1532-1543.

[3] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP*, 3982-3992. DOI: 10.18653/v1/D19-1410.

[4] Grover, A., and Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. *Proceedings of KDD*, 855-864. DOI: 10.1145/2939672.2939754.